Expt. No. 6

Title: Apply Mini Batch GD on a supervised single layer feed forward neural network.

Rubric	***Score (0 to 5)***
Knowledge	
Delivery	
Output and Conclusion	
Total	

Performed On: 01/09/2025

Sign: _____

Experiment 6

Aim: Apply Mini-batch gradient descent algorithms to learn the parameters of a supervisedsingle layer feed forward network.

Software: Google Collab

Prelab Question:

Explain the use of gradient descent algorithm in learning the parameters?

Gradient descent is used to optimize model parameters by minimizing the loss function through iterative updates.

Post Lab Question:

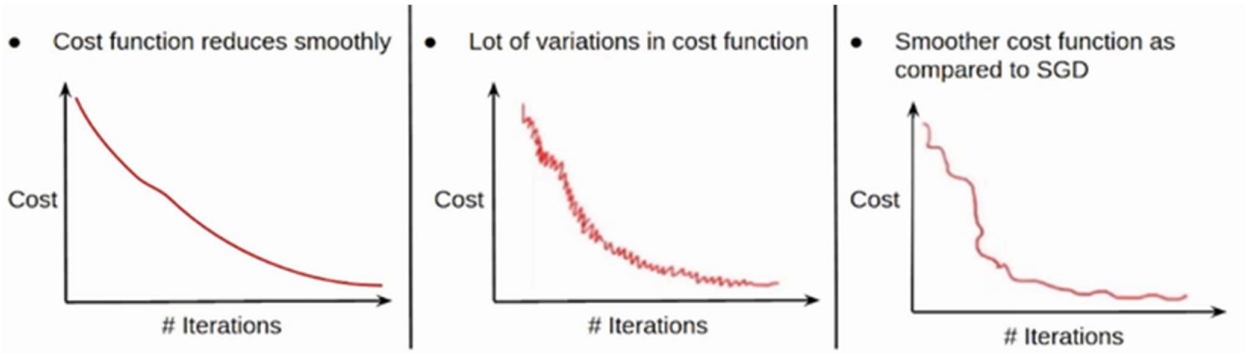
Explain various types of gradient descent algorithms?

Gradient descent types are batch, stochastic, and mini-batch, based on the number of samples used per update.

Theory:

In machine learning, gradient descent is an optimization technique used for computing the model parameters (coefficients and bias) for algorithms like linear regression, logistic regression, neural networks, etc. In this technique, we repeatedly iterate through the training set and update the model parameters in accordance with the gradient of the error with respect to the training set. Depending on the number of training examples considered in updating the model parameters, we have 3-types of gradient descents:

1. **Batch Gradient Descent:** Parameters are updated after computing the gradient of the error with respect to the entire training set
2. **Stochastic Gradient Descent:** Parameters are updated after computing the gradient of the error with respect to a single training example
3. **Mini-Batch Gradient Descent:** Parameters are updated after computing the gradient of the error with respect to a subset of the training set
4. Thus, mini-batch gradient descent makes a compromise between the speedy convergence and the noise associated with gradient update which makes it a more flexible and robust algorithm.



Algorithm:

Let theta = model parameters and max_iters = number of epochs. for itr = 1, 2, 3, …, max_iters: for mini_batch (X_mini, y_mini):

- Forward Pass on the batch X_mini:
  - Make predictions on the mini-batch
  - Compute error in predictions (J(theta)) with the current values of the parameters
- Backward Pass:
  - Compute gradient(theta) = partial derivative of J(theta) w.r.t. theta
- Update parameters:
  - theta = theta – learning_rate*gradient(theta)

Program & Output

```
# importing dependencies
import numpy as np
import matplotlib.pyplot as plt

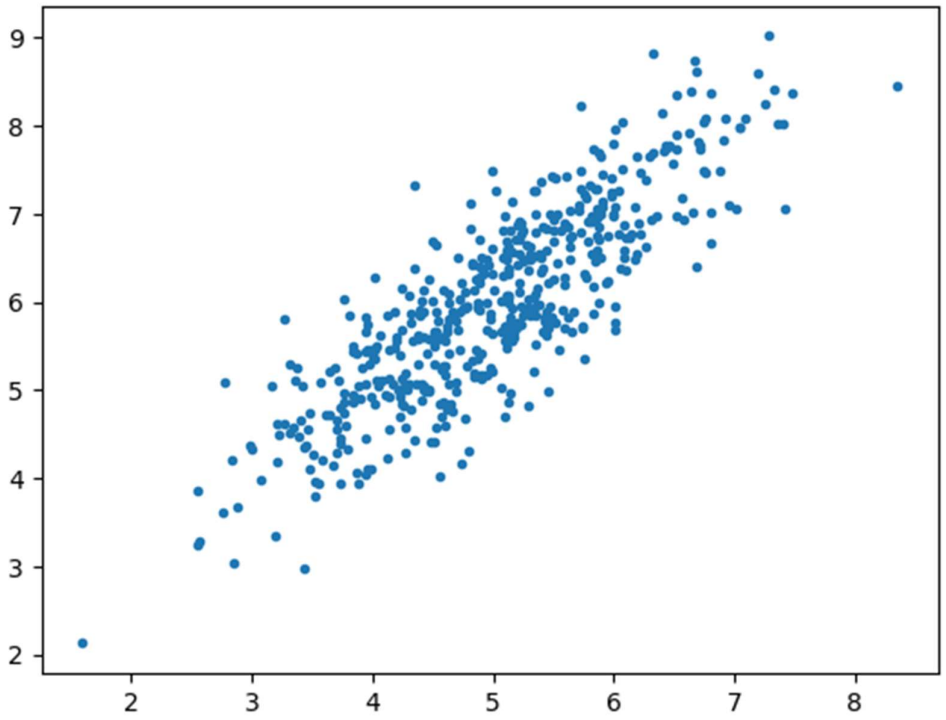
# creating data
mean = np.array([5.0, 6.0])
cov = np.array([[1.0, 0.95], [0.95, 1.2]])
data = np.random.multivariate_normal(mean, cov, 6000)

# visualising data
plt.scatter(data[:500, 0], data[:500, 1], marker='.')
plt.show()

# train-test-split
data = np.hstack((np.ones((data.shape[0], 1)), data))

split_factor = 0.90
split = int(split_factor * data.shape[0])

X_train = data[:split, :-1]
y_train = data[:split, -1].reshape((-1, 1))
X_test = data[split:, :-1]
y_test = data[split:, -1].reshape((-1, 1))
```




```
# linear regression using "mini-batch" gradient descent
# function to compute hypothesis / predictions

def hypothesis(X, theta):
  return np.dot(X, theta)

# function to compute gradient of error function w.r.t. theta

def gradient(X, y, theta):
  h = hypothesis(X, theta)
  grad = np.dot(X.transpose(), (h - y))
  return grad

# function to compute the error for current values of theta

def cost(X, y, theta):
  h = hypothesis(X, theta)
  J = np.dot((h - y).transpose(), (h - y))
  J /= 2
  return J[0]

# function to create a list containing mini-batches

def create_mini_batches(X, y, batch_size):
  mini_batches = []
  data = np.hstack((X, y))
  np.random.shuffle(data)
  n_minibatches = data.shape[0] // batch_size
```

```
  i = 0

  for i in range(n_minibatches + 1):
    mini_batch = data[i * batch_size:(i + 1)*batch_size, :]
    X_mini = mini_batch[:, :-1]
    Y_mini = mini_batch[:, -1].reshape((-1, 1))
    mini_batches.append((X_mini, Y_mini))
  if data.shape[0] % batch_size != 0:
    mini_batch = data[i * batch_size:data.shape[0]]
    X_mini = mini_batch[:, :-1]
    Y_mini = mini_batch[:, -1].reshape((-1, 1))
    mini_batches.append((X_mini, Y_mini))
  return mini_batches

# function to perform mini-batch gradient descent

def gradientDescent(X, y, learning_rate=0.001, batch_size=32):
  theta = np.zeros((X.shape[1], 1))
  error_list = []
  max_iters = 3
  for itr in range(max_iters):
    mini_batches = create_mini_batches(X, y, batch_size)
    for mini_batch in mini_batches:
      X_mini, y_mini = mini_batch
      theta = theta - learning_rate * gradient(X_mini, y_mini, theta)
      error_list.append(cost(X_mini, y_mini, theta))

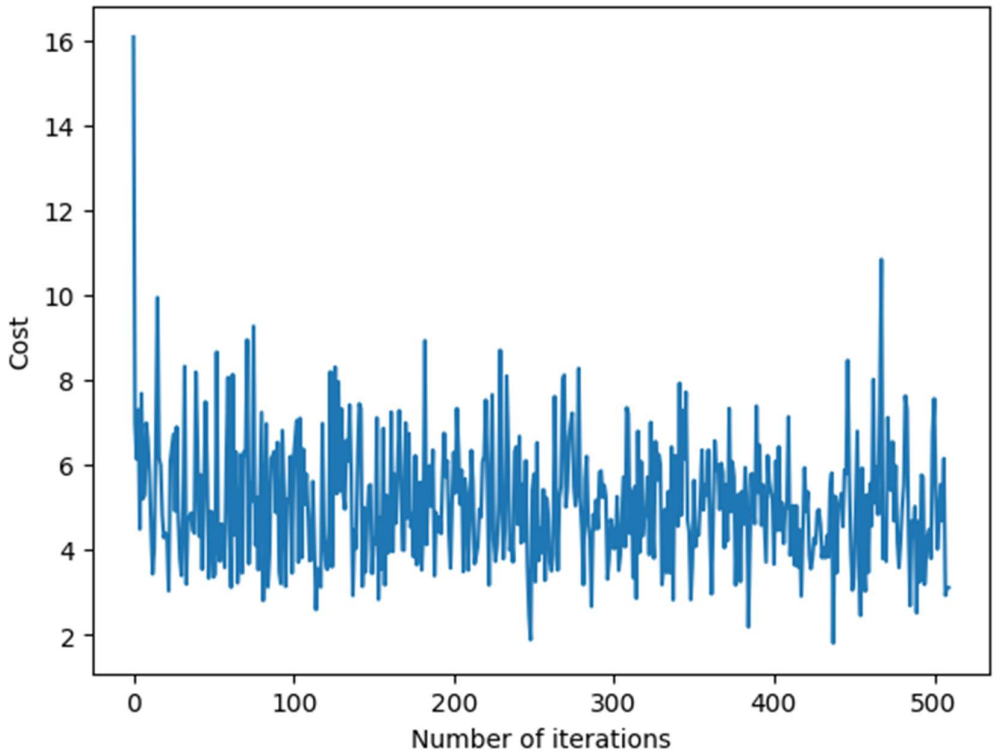
  return theta, error_list


theta, error_list = gradientDescent(X_train, y_train)
print("Bias = ", theta[0])
print("Coefficients = ", theta[1:])

# visualising gradient descent
plt.plot(error_list)
plt.xlabel("Number of iterations")
plt.ylabel("Cost")
plt.show()
```

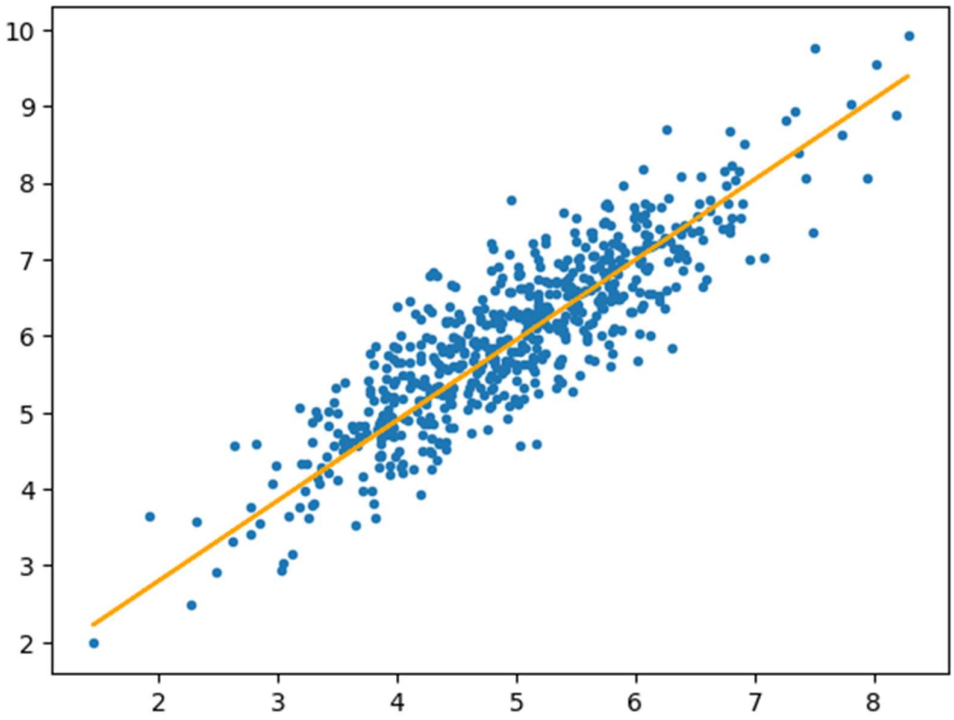
OUTPUT:

```
Bias =  [0.69901842]
Coefficients =  [[1.04894801]]
```




```
# predicting output for X_test
y_pred = hypothesis(X_test, theta)
plt.scatter(X_test[:, 1], y_test[:, ], marker='.')
plt.plot(X_test[:, 1], y_pred, color='orange')
plt.show()

# calculating error in predictions
error = np.sum(np.abs(y_test - y_pred) / y_test.shape[0])
print("MSE=",error)
```



MSE= 0.454571665257975

Conclusion:

Applying Mini-Batch Gradient Descent to a supervised single-layer feedforward neural network balances the efficiency of batch processing with the robustness of stochastic updates. It enhances training stability and convergence speed for better model performance.

CO's Covered:

CO1. Implement basic neural network models.
CO2. Design and train feedforward neural networks using various learning algorithms and optimize model performance.